



# Topological Sorting and Connected Components

Data Structures & Algorithms

Week 8

## ► Topological Sorting and Connected Components

Sagnik Mukhopadhyay

s.mukhopadhyay@bham.ac.uk

<https://sagnikm.github.io/>

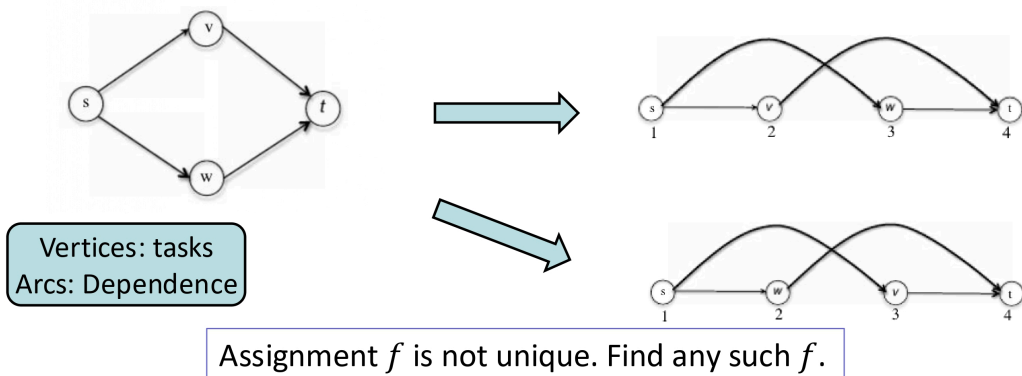
## ► Aims of this week

- We've seen BFS on graphs.
  - Useful in finding connectivity and distances<sup>TC</sup>.
- Now: **Graph Sorting and Strong Connectivity.**
  - Strong connectivity: Older brother of connectivity.
  - Graph sorting: What are we comparing?
- Same tool to solve both. You have seen it already!

## ► Topological Sorting/Ordering

## ► What is a topological sorting/ordering?

- **Given:** Directed graph  $G = (V, E)$ .
- **Goal:** Come up with assignment  $f$  of numbers on  $V$  such that:
  - For each arc  $(v, w) \in E, f(v) < f(w)$ .

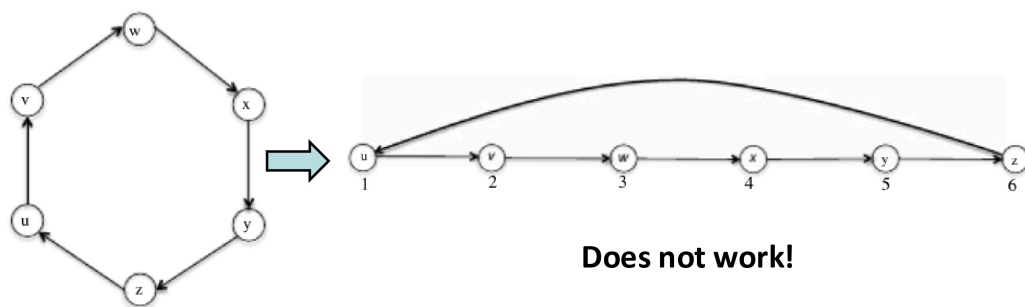


Data Structure & Algorithms

4

## ► Can we always sort topologically?

- **Given:** Directed graph  $G = (V, E)$ .
- **Goal:** Come up with assignment  $f$  of numbers on  $V$  such that:
  - For each arc  $(v, w) \in E, f(v) < f(w)$ .

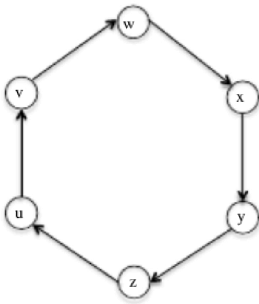


Data Structure & Algorithms

5

## ► Can we always sort topologically?

- **Given:** Directed graph  $G = (V, E)$ .
- **Goal:** Come up with assignment  $f$  of numbers on  $V$  such that:
  - For each arc  $(v, w) \in E, f(v) < f(w)$ .



Is this the only type of bad graph?

- Yes-ish! Any graph that contains a directed cycle is bad.
- **Important:** Directed acyclic graphs are always good.
  - Can you prove why?

Data Structure & Algorithms

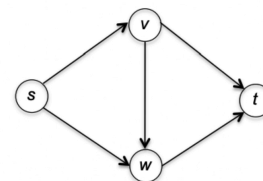
6

## ► Go wide or go deep?

- **Given:** Directed **acyclic** graph  $G = (V, E)$ .
- **Goal:** Come up with assignment  $f$  of numbers on  $V$  such that:
  - For each arc  $(v, w) \in E, f(v) < f(w)$ .

**How about our friend BFS?** Number vertices in order they are explored!

What goes wrong?



(b) A directed graph

Data Structure & Algorithms

7



## ► Go deep instead

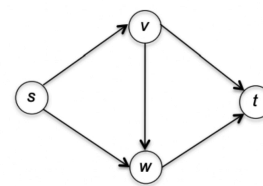
- **Given:** Directed **acyclic** graph  $G = (V, E)$ .
- **Goal:** Come up with assignment  $f$  of numbers on  $V$  such that:
  - For each arc  $(v, w) \in E, f(v) < f(w)$ .

Set global variable  $\text{currentLabel} = n$ , **Explored** =  $\emptyset$ , **Unexplored** =  $V$ .

**TopoSort( $G, s$ ):**

- Mark  $s$  as **Explored**.
- For each outgoing arc  $(s, v)$ , do:
  - If  $v$  is **Unexplored**: **TopoSort( $G, v$ )**
- $f(s) = \text{currentLabel}$
- $\text{currentLabel} = \text{currentLabel} - 1$ .

Explored each edge once.  $O(m + n)$ .



(b) A directed graph

## ► Correctness

Set global variable  $\text{currentLabel} = n$ , **Explored** =  $\emptyset$ , **Unexplored** =  $V$ .

**TopoSort( $G, s$ ):**

- Mark  $s$  as **Explored**.
- For each outgoing arc  $(s, v)$ , do:
  - If  $v$  is **Unexplored**: **TopoSort( $G, v$ )**
- $f(s) = \text{currentLabel}$
- $\text{currentLabel} = \text{currentLabel} - 1$ .

Consider an arbitrary edge  $(v, w)$ : Need to ensure  $f(v) < f(w)$ .

If  $v$  is discovered first, then  $f(w)$  is assigned before  $f(v) \Rightarrow f(v) < f(w)$ .

If  $w$  is discovered first,  $v$  cannot be discovered in **TopoSort( $G, w$ )**. **Why?**

**Hence,**  $v$  is discovered after **TopoSort( $G, w$ )**  $\Rightarrow f(v) < f(w)$ .

## ► Depth-first search/exploration on graphs

Set global variable  $\text{currentLabel} = n$ ,  $\text{Explored} = \emptyset$ ,  $\text{Unexplored} = V$ .

**TopoSort** DFS( $G, s$ ):

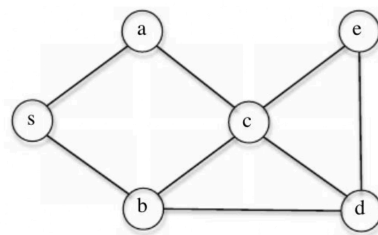
- Mark  $s$  as **Explored**.
- For each outgoing arc  $(s, v)$ , do:
  - If  $v$  is **Unexplored**: **TopoSort** DFS( $G, v$ )
- $f(s) = \text{currentLabel}$
- $\text{currentLabel} = \text{currentLabel} - 1$ .

## ► DFS: Two versions

Set  $\text{Explored} = \emptyset$ ,  $\text{Unexplored} = V$ .

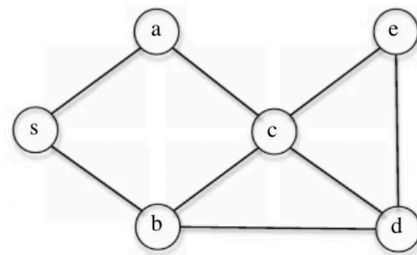
**Rec-DFS**( $G, s$ ):

- Mark  $s$  as **Explored**.
- For each outgoing arc  $(s, v)$ , do:
  - If  $v$  is **Unexplored**: **Rec-DFS**( $G, v$ )



**Iterative-DFS**( $G, s$ ):

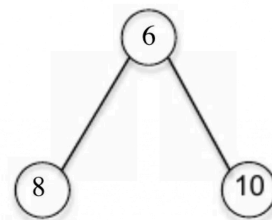
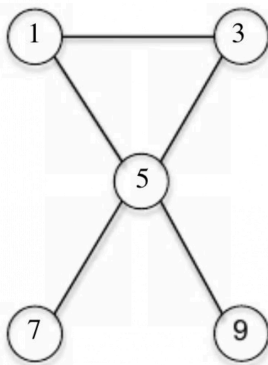
- $S$  = A stack initialised with  $s$ .
- While  $S$  is not empty, do:
  - Pop  $u$  from  $S$  and mark  $u$  as **Explored**.
  - For each outgoing arc  $(u, v)$ , do:
    - If  $v$  is **Unexplored**: Push  $v$  is  $S$ .



## ► Strongly Connected Component

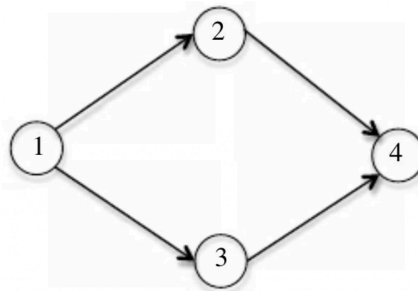
## ► What is Connected Component?

- Given a graph  $G = (V, E)$ .
- Connected component is a maximal set of vertices  $S$  such that:
  - Any vertex in  $S$  can reach any other vertex in  $S$ .



## ► What is Connected Component?

- Given a graph  $G = (V, E)$ .
- Connected component is a maximal set of vertices  $S$  such that:
  - Any vertex in  $S$  can reach any other vertex in  $S$ .
- What happens when the graph is directed?

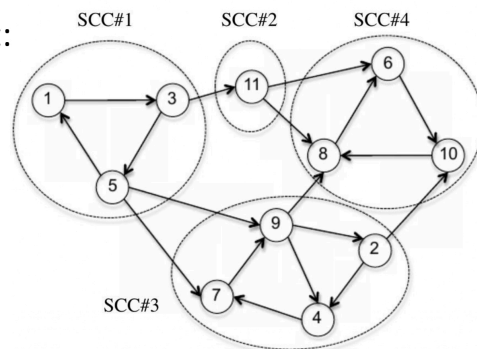


Data Structure & Algorithms

14

## ► Strongly Connected Component

- Given a graph  $G = (V, E)$ .
- Connected component is a maximal set of vertices  $S$  such that:
  - Any vertex in  $S$  can reach any other vertex in  $S$ .
- What happens when the graph is directed?
- Strongly connected component:
  - Same definition.
  - **But edge can exist between SCCs.**
  - **Only for directed graphs!**



Data Structure & Algorithms

15

## ► Easy: Undirected Connected Component

- **Input:** an **undirected** graph  $G = (V, E)$ .
- **Goal:** Identify all connected components.

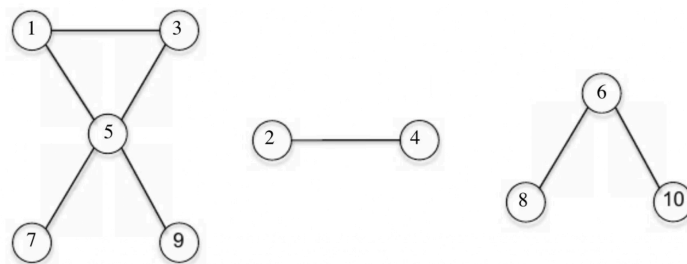
- Set all vertices as **Unexplored**
- Set CCNumber = 0
- While there is an **Unexplored** vertex  $v$ , do:
  - CCNumber = CCNumber + 1
  - Call  $\text{BFS}(G, v)$  and assign all **Explored** vertices CCNumber.

Data Structure & Algorithms

16

## ► Easy: Undirected Connected Component

- Set all vertices as **Unexplored**
- Set CCNumber = 0 **Runtime:  $O(m + n)$**
- While there is an **Unexplored** vertex  $v$ , do:
  - CCNumber = CCNumber + 1
  - Call  $\text{BFS}(G, v)$  and assign all **Explored** vertices CCNumber.

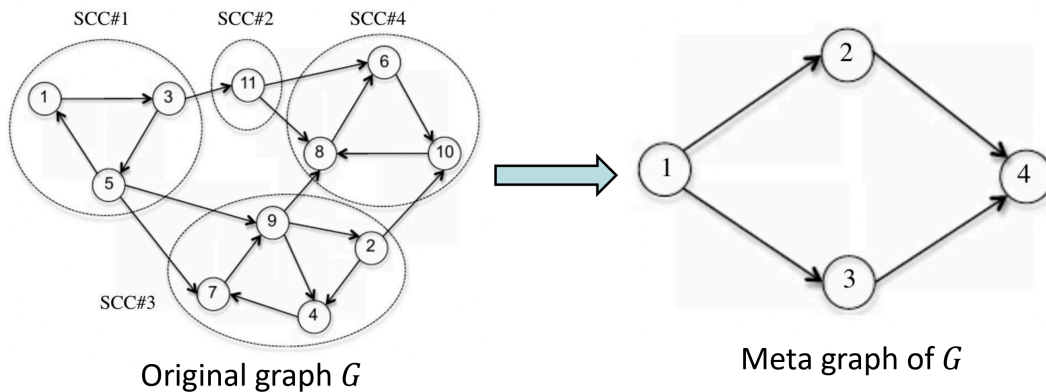


Data Structure & Algorithms

17

## ► Tricky: Strongly Connected Component

- **Input:** a **directed** graph  $G = (V, E)$ .
- **Goal:** Identify all strongly connected components.



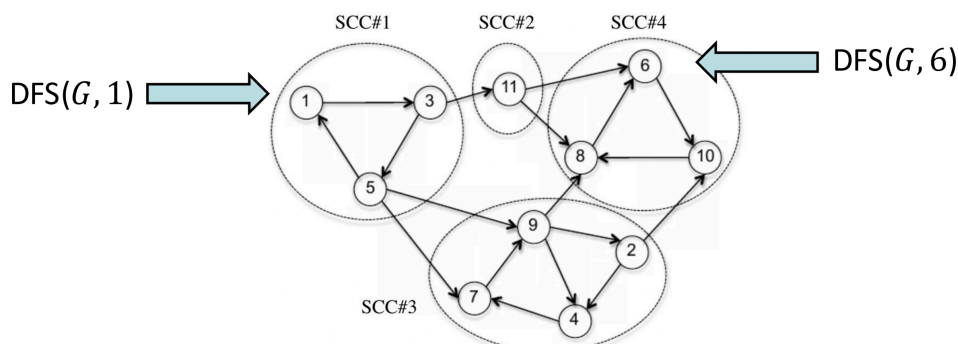
**Observe:** SCC Meta graph is acyclic.

Data Structure & Algorithms

18

## ► SCC: Does DFS Help?

- **Input:** a **directed** graph  $G = (V, E)$ .
- **Goal:** Identify all strongly connected components.



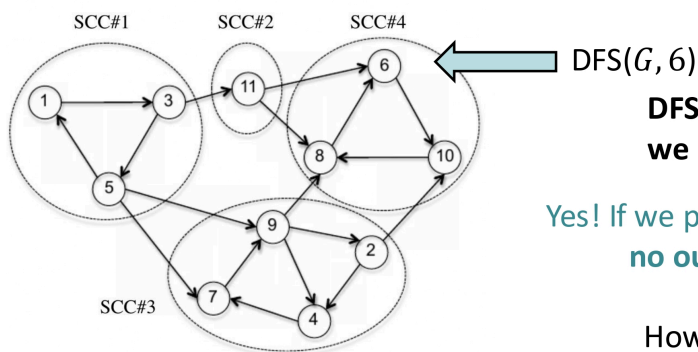
**DFS can help sometimes! Can we be more precise?**

Yes! If we pick a vertex from a sink SCC, i.e., **no outgoing arc from that SCC.**

Data Structure & Algorithms

19

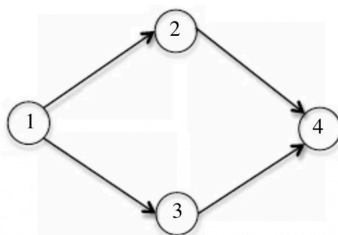
## ► SCC: Does DFS Help?



DFS can help sometimes! Can we be more precise?

Yes! If we pick a vertex from a sink SCC, i.e., **no outgoing arc from that SCC.**

How do find such a sink SCC?



Meta graph of  $G$

Data Structure & Algorithms

### Plan!

- Topological sort of the meta graph has 4 in the end.
- Maybe we should do a topological sort first.
- Then find SCCs in reverse top order.

20

## ► SCC: Kosaraju's Algorithm

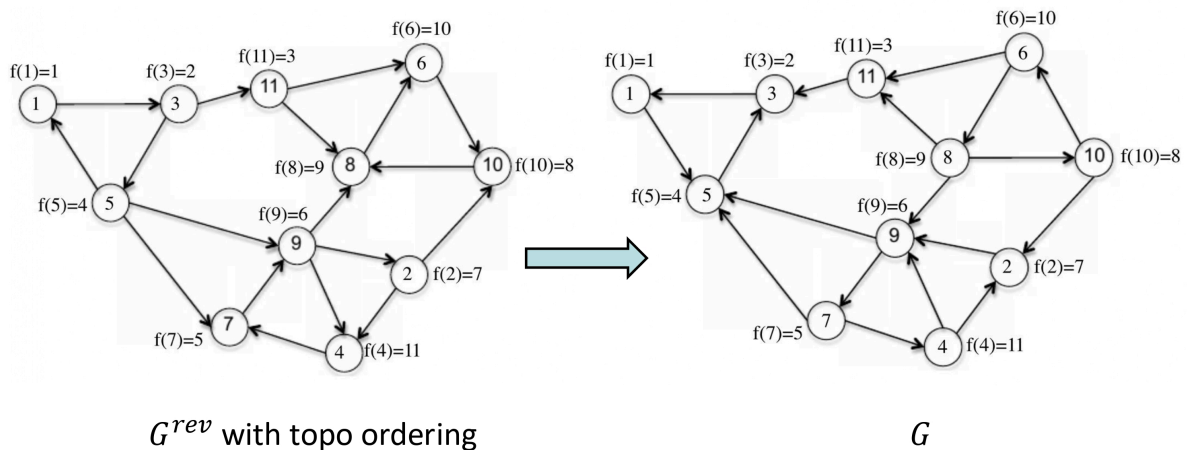
- $G^{rev} = G$  with all edge directions reversed.
- Mark all vertices of  $G^{rev}$  Unexplored.
- $\text{TopoSort}(G^{rev}, s)$   $s$  is any arbitrary vertex

- Mark all vertices of  $G$  Unexplored.
- Set  $\text{SCCNumber} = 0$
- For each  $v \in V$  in increasing order of  $f(v)$ , do:
  - If  $v$  is Unexplored:
    - $\text{SCCNumber} = \text{SCCNumber} + 1$
    - $\text{DFS}(G, v)$

Data Structure & Algorithms

21

## ► SCC: Kosaraju's Algorithm in action



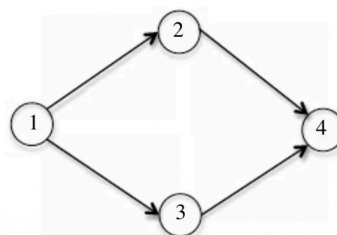
## ► Kosaraju's Algorithm: Why does it work?

- $G^{rev} = G$  with all edge directions reversed.
- Mark all vertices of  $G^{rev}$  Unexplored.
- $\text{TopoSort}(G^{rev}, s)$   $s$  is any arbitrary vertex

Why consider  $G^{rev}$ ? What happens if  $\text{TopoSort}(G, s)$ ?

### Remember:

- Our plan was to find a vertex in a sink SCC.
- We thought a **topological order** will have vertices of a sink SCC in the **end**.
- And **this is true for DAG (meta graph)**.
- Is it true on the original graph?

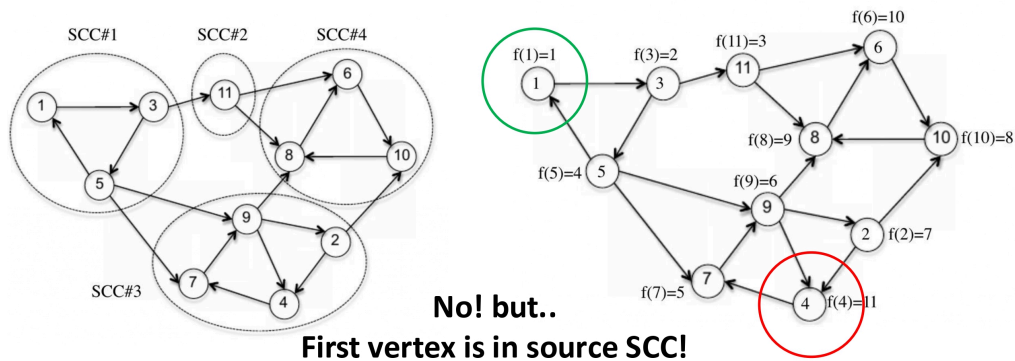




## ► Kosaraju's Algorithm: Why does it work?

- $G^{rev} = G$  with all edge directions reversed.
- Mark all vertices of  $G^{rev}$  Unexplored.
- $\text{TopoSort}(G^{rev}, s)$   $s$  is any arbitrary vertex

Why consider  $G^{rev}$ ? What happens if  $\text{TopoSort}(G, s)$ ?



Data Structure & Algorithms

24

## ► Kosaraju's Algorithm: Why does it work?

- $G^{rev} = G$  with all edge directions reversed.
- Mark all vertices of  $G^{rev}$  Unexplored.
- $\text{TopoSort}(G^{rev}, s)$   $s$  is any arbitrary vertex

Why consider  $G^{rev}$ ? What happens if  $\text{TopoSort}(G, s)$ ?

### Theorem:

- $f$  = Topo ordering on  $G$ ,
  - Two SCCs  $S_1, S_2$  with edge  $(v, w)$  such that  $v \in S_1$  and  $w \in S_2$ ,
- Then:

$$\min_{x \in S_1} f(x) < \min_{y \in S_2} f(y).$$

Data Structure & Algorithms

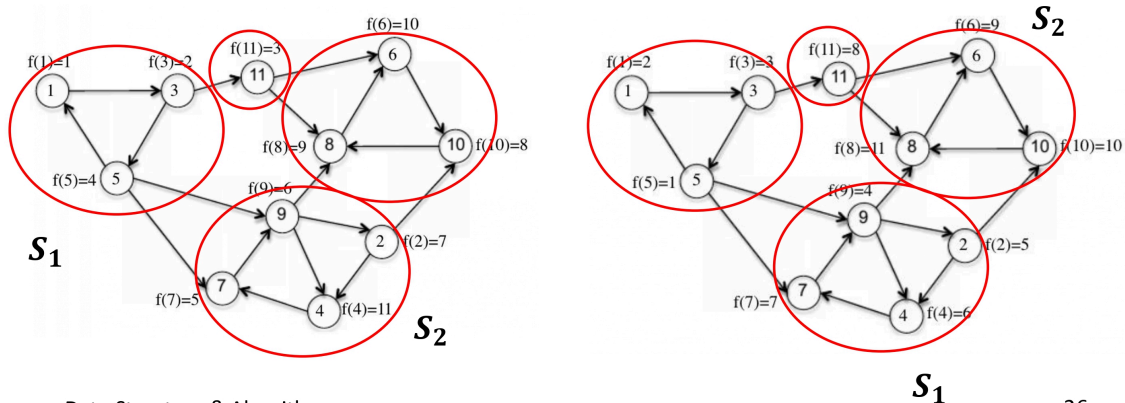
25

## ► Kosaraju's Algorithm: Why does it work?

### Theorem:

- $f$  = Topo ordering on  $G$ ,
  - Two SCCs  $S_1, S_2$  with edge  $(v, w)$  such that  $v \in S_1$  and  $w \in S_2$ ,
- Then:

$$\min_{x \in S_1} f(x) < \min_{y \in S_2} f(y).$$



Data Structure & Algorithms

26

## ► Kosaraju's Algorithm: Why does it work?

### Theorem:

- $f$  = Topo ordering on  $G$ ,
  - Two SCCs  $S_1, S_2$  with edge  $(v, w)$  such that  $v \in S_1$  and  $w \in S_2$ ,
- Then:

$$\min_{x \in S_1} f(x) < \min_{y \in S_2} f(y).$$

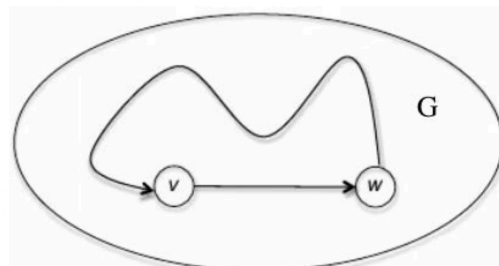
Value of the first vertex

### Recall: Correctness of TopoSort

Need to ensure  $f(v) < f(w)$ .

If  $v$  is discovered first, then  $f(w)$  is assigned before  $f(v) \Rightarrow f(v) < f(w)$ .

If  $w$  is discovered first,  $v$  cannot be discovered in  $\text{TopoSort}(G, w)$ . **Why?**



Data Structure & Algorithms

27

## ► Kosaraju's Algorithm: Why does it work?

### Theorem:

- $f$  = Topo ordering on  $G$ ,
- Two SCCs  $S_1, S_2$  with edge  $(v, w)$  such that  $v \in S_1$  and  $w \in S_2$ ,

Then:

$$\min_{x \in S_1} f(x) < \min_{y \in S_2} f(y).$$

Value of the first vertex

If  $S_1$  is discovered first, then

$$\min_{x \in S_1} f(x) < \min_{y \in S_2} f(y).$$

If  $S_2$  is discovered first, no vertex in  $S_1$  cannot be discovered in  $\text{TopoSort}(G, w \in S_2)$ . **Why?**

$S_1$  and  $S_2$  will not be maximal then!

